Python 3 | recto · le langage

mot-clé type intégré # commentaire «à remplir»

</> Opérateurs

1 · bases

Arithm.	`+ - * / // % **`	`/` réel, `//` entier, `**` puissance
Compar.	`== != < <= > >=`	chaînables : `0 <= i < n`
Logiques	`and or not`	court-circuit, rendent un opérande
Identité	`is, is not`	même objet (pour `None`)
Appart.	`in, not in`	liste, chaîne, clé de dict
Bits	`& | ^ ~ << >>`	sur les `int`
Ternaire	`a if c else b`	une expression
Morse	`(n := len(x))`	affecte dans une expression 3.8

Types et nombres

```
n = 42              # int, sans limite de taille
x = 3.14            # float : double IEEE 754
z = 2 + 3j          # complex
ok = True           # bool (sous-classe d'int)
s = "texte"         # str : Unicode, immuable
b = b"\x01\xff"     # bytes
r = None            # absence de valeur
type(x), isinstance(x, (int, float))
```

Immuables : `int float str tuple bytes frozenset`. Mutables : `list dict set` et la plupart des objets.

Nombres

```
int("ff", 16), int("0b101", 0)  # 255, 5
hex(255), bin(5), oct(8)        # '0xff', '0b101', '0o10'
-7 // 2, -7 % 2, divmod(17, 5)  # -4, 1, (3, 2)
round(2.5), round(3.14159, 2)   # 2 (au pair), 3.14
int(3.9), abs(-2), 2 ** 0.5     # 3 (tronque), 2, 1.414
1_000_000, 1e-9, float("inf")
import math                     # pi, sqrt, log2, ceil...
```

Chaînes et f-strings

```
  0  1  2  3  4  5  6
 | P | y | t | h | o | n |
-6 -5 -4 -3 -2 -1
```

`s[1:4]` → `'yth'` (fin exclue)
`s[-2:]` → `'on'`

```
s[0], s[-1], s[1:4], s[::-1]      # [::-1] : inverse
"a,b,c".split(",")                # ['a', 'b', 'c']
", ".join(["a", "b"])             # 'a, b'
s.strip(), s.lower(), s.upper()
s.replace("a", "b"), s.find("x")  # -1 si absent
s.startswith("x"), "ab" in s
f"{x:.3f} {n:5d} {n:#06x}"        # '3.140    42 0x002a'
f"{s:>10} {s:<10} {s:^10}"        # aligner sur 10
f"{x=}"                           # 'x=3.14' (débogage)
r"C:\new"                         # brute : \ non échappé
```

>_ Console

```
print(a, b, sep=", ", end="")  # sans retour ligne
print(f"{nom:<8}{v:6.2f}")     # colonnes alignées
nom = input("nom ? ")          # toujours une str
n = int(input("n ? "))
import sys; sys.argv, sys.exit(1)  # arguments, code
```

Fichiers et arguments nommés : verso (colonne 4).

Contrôle et boucles

2 · contrôle · fonctions

```
if x > 0:
    ...
elif x == 0:
    ...
else:
    ...
while cond:
    if fini: break  # sort de la boucle
    continue        # tour suivant
for i in range(début, fin, pas): ...
for i, x in enumerate(l, start=1): ...
for a, b in zip(l1, l2): ...  # s'arrête au plus court
for x in reversed(l): ...
any(x > 0 for x in l), all(l)
min(l, key=len), max(d, key=d.get)
match cmd.split():  # filtrage de motifs (3.10)
    case ["go", dest]: aller(dest)
    case ["stop"] | ["q"]: quitter()
    case _: print("inconnu")
```

Les blocs sont définis par l'indentation (4 espaces). `pass` = bloc vide.

Fonctions

```
def f(a, b=2, *args, k=0, **kw) -> int:
    """Docstring : ce que fait f."""
    return a + b                  # None si pas de return
f(1); f(1, 3, 4, 5, k=1, x=9)     # args=(4, 5), kw={'x': 9}
f(*liste, **dico)                 # dépaqueter à l'appel
carre = lambda x: x * x           # fonction anonyme
def g(x: float, y: list[int] | None = None): ...
```

Paramètres après `*args` (ou `*`) : seulement par mot-clé. Les annotations ne sont pas vérifiées à l'exécution. Affecter une variable dans une fonction la rend locale : `global` / `nonlocal` pour modifier celle du dessus.

⚠Piège : `def f(l=[])` garde la même liste d'un appel à l'autre : défaut `None`, puis `l = l or []`.

Exceptions

```
try:
    v = int(txt)
except (ValueError, TypeError) as e:
    print("invalide :", e)
else:
    ...  # si aucune exception
finally:
    ...  # toujours exécuté
raise ValueError(f"mauvais n : {n}")
class SimError(Exception): pass
```

`with` ferme le fichier même en cas d'exception.

Modules et imports

```
import numpy as np        # alias
from pathlib import Path  # un nom
from . import utils       # relatif (paquet)
if __name__ == "__main__": main()
```

Fichier = module ; dossier avec `__init__.py` = paquet.

Listes et tuples

3 · collections · classes

```
l = [3, 1, 2]
l.append(4); l.extend([5, 6])  # en place
l.insert(0, 9); l.pop(); l.pop(0)
l.remove(3)                    # 1re occurrence
l.sort(key=abs, reverse=True)  # en place, rend None
sorted(l)                      # nouvelle liste triée
l.index(2), l.count(2), len(l)
l[1:3] = []                    # suppr. une tranche
t = (1, "a")                   # tuple : immuable
a, *reste = [1, 2, 3]          # a=1, reste=[2, 3]
a, b = b, a                    # échange
```

⚠Piège : `b = a` ne copie pas : `a.copy()`, ou `copy.deepcopy(a)` si imbriqué.

Dictionnaires et ensembles

```
d = {"clk": 10, "rst": 0}
d["en"] = 1                    # ajout / mise à jour
d.get("x", 0)                  # défaut si absente
d.setdefault("k", []).append(1)
for k, v in d.items(): ...     # .keys() .values()
d.pop("rst"); d | {"a": 1}     # retirer, fusionner
"clk" in d                     # test de clé
e = {1, 2, 3}; e.add(4)
e | f, e & f, e - f, e ^ f     # union, inter, diff
from collections import defaultdict, Counter
cnt = Counter("abracadabra")   # {'a': 5, 'b': 2...}
grp = defaultdict(list)        # grp[k].append(v)
```

Clés : objets hachables (immuables). L'ordre d'insertion est conservé.

Compréhensions

```
[x**2 for x in range(10) if x % 2]  # liste
[x if x > 0 else 0 for x in l]      # avec ternaire
{k: v for k, v in zip(noms, vals)}  # dict
{c.lower() for c in s}              # ensemble
sum(x**2 for x in data)             # générateur
[[0] * n for _ in range(m)]         # matrice m x n
[x for ligne in m for x in ligne]   # aplatir
```

Classes

```
class Reg:
    nb = 0                  # attribut de classe
    def __init__(self, nom, w=8):
        self.nom, self.w = nom, w
        Reg.nb += 1
    def __repr__(self):
        return f"Reg({self.nom!r}, {self.w})"
    @property
    def maxi(self): return 2**self.w - 1
class Compteur(Reg):  # héritage
    def __init__(self, nom):
        super().__init__(nom, 16)
```

Python 3 | verso · scripts d'ingénieur

mot-clé type intégré # commentaire «à remplir»

>_ Fichiers texte

4 · fichiers · scripts

```
with open("in.txt", encoding="utf-8") as fh:
    for ligne in fh:           # ligne par ligne
        ligne = ligne.rstrip("\n")
with open("out.txt", "w") as fh:  # "a" = ajouter
    fh.write(f"{n}\n")
    print("ok", file=fh)
txt = Path(f).read_text()      # tout, fermé aussitôt
lignes = txt.splitlines()
data = Path(f).read_bytes()    # binaire
```

Modes : `"r"` lecture, `"w"` écrase, `"a"` ajoute, `"b"` binaire (bytes).

>_ Chemins (pathlib)

```
from pathlib import Path
p = Path("runs") / "synth" / "area.rpt"
p.exists(), p.is_file(), p.is_dir()
p.name, p.stem, p.suffix  # area.rpt area .rpt
p.parent, p.with_suffix(".csv")
p.read_text(); p.write_text("...")
Path("out").mkdir(parents=True, exist_ok=True)
for f in Path("runs").glob("*/area.rpt"): ...
for f in Path(".").rglob("*.v"): ...  # récursif
Path.cwd(), Path.home(), p.resolve()
```

>_ CSV et JSON

```
import csv, json
with open("res.csv", newline="") as fh:
    for row in csv.DictReader(fh):  # dict par ligne
        f = float(row["freq"])      # valeurs en str
with open("out.csv", "w", newline="") as fh:
    w = csv.writer(fh)
    w.writerow(["design", "area"])
    w.writerows(lignes)
cfg = json.loads(Path("cfg.json").read_text())  # dict
Path("o.json").write_text(json.dumps(cfg, indent=2))
import pandas as pd
df = pd.read_csv("res.csv"); df["area"].mean()
```

Script : argparse, main

```
#!/usr/bin/env python3
"""Rassemble les rapports de synthèse."""
import argparse, sys
def main() -> int:
    ap = argparse.ArgumentParser(description=__doc__)
    ap.add_argument("dossier")  # positionnel
    ap.add_argument("-o", "--out", default="res.csv")
    ap.add_argument("-n", type=int, default=10)
    ap.add_argument("-v", action="store_true")
    args = ap.parse_args()      # args.out...
    return 0
if __name__ == "__main__":      # pas à l'import
    sys.exit(main())
```

`python3 script.py runs -o a.csv -v` ; `-h` est généré.

Lancer des outils (subprocess)

5 · outils

```
import subprocess, os, shutil
r = subprocess.run(["vsim", "-c", "-do", "run.do"],
                   capture_output=True, text=True,
                   cwd="sim", timeout=600)
r.returncode, r.stdout, r.stderr
subprocess.run(cmd, check=True)  # exception si échec
os.environ.get("HOME"); os.environ["SEED"] = "3"
shutil.copy(src, dst); shutil.rmtree(dossier)
shutil.which("vivado")           # chemin ou None
```

Passer une liste d'arguments plutôt qu'une chaîne avec `shell=True` : pas de souci d'espaces ni d'injection.

Expressions régulières (re)

```
import re
m = re.search(r"Total area:\s+([\d.]+)", txt)
if m: area = float(m.group(1))
re.findall(r"\d+", "a1 b22 c333")  # ['1','22','333']
re.sub(r"\s+", " ", txt)           # remplacer
pat = re.compile(r"(?P<sig>\w+)\s*=\s*(?P<v>\d+)")
for m in pat.finditer(txt):
    print(m["sig"], int(m["v"]))
```

`. \d \w \s`	tout caractère, chiffre, mot, blanc
`* + ? {n,m}`	0+, 1+, 0 ou 1, de n à m (`*?` non gourmand)
`^ $ \b`	début, fin, limite de mot
`[a-f] (...)`	classe de caractères, groupe capturant

Toujours des chaînes brutes `r"..."` pour les motifs.

Exemple : rapports de synthèse en CSV

```
import csv, re
from pathlib import Path
RE_AREA = re.compile(r"Total cell area:\s+([\d.]+)")
RE_PWR = re.compile(r"Total Dynamic Power\s+=\s+([\d.]+)")
rows = []
for rpt in sorted(Path("runs").glob("*/report.txt")):
    txt = rpt.read_text()
    a, p = RE_AREA.search(txt), RE_PWR.search(txt)
    if not (a and p):
        print(f"incomplet : {rpt}"); continue
    rows.append([rpt.parent.name, float(a[1]),
                 float(p[1])])
with open("bilan.csv", "w", newline="") as fh:
    w = csv.writer(fh)
    w.writerow(["config", "area_um2", "power_mW"])
    w.writerows(rows)
```

Modules utiles

math	`sqrt log2 ceil pi`	`isclose(a, b)`
statistics	`mean median stdev`	sur une liste
itertools	`product combinations`	`chain groupby`
random	`seed randint choice`	`random()` dans $[0, 1[$
time	`perf_counter sleep`	chronométrer
datetime	`datetime.now()`	`.strftime("%H:%M")`
logging	`basicConfig info`	plutôt que `print`

numpy

6 · numpy · tracés · pièges

```
import numpy as np
a = np.array([1, 2, 3], dtype=float)
np.zeros((3, 4)), np.ones(5), np.eye(3)
np.arange(0, 1, 0.1), np.linspace(0, 1, 11)
a.shape, a.dtype, a.reshape(3, 1), a.T
a * 2 + 1, a @ b, np.sqrt(a)  # élément par élément
m[1, :], m[:, 0], m[m > 0]    # tranche, masque
a.mean(), a.std(), a.max(), a.argmax()
m.sum(axis=0)                  # somme par colonne
d = np.loadtxt("w.csv", delimiter=",", skiprows=1)
np.savetxt("o.csv", d, delimiter=",", fmt="%.4g")
```

Les tranches sont des vues (pas des copies) : `a.copy()` pour copier.
Broadcasting : `(3,1) + (4,)` donne `(3,4)`.

matplotlib

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(6, 3.5))
ax.plot(t, v, label="v(t)")  # courbe
ax.semilogy(f, err, "o--")   # log en y
ax.set_xlabel("t (ns)"); ax.set_ylabel("V")
ax.grid(True); ax.legend()
fig.tight_layout(); fig.savefig("v.pdf")
plt.show()                   # fenêtre
```

Bits et registres

```
(x >> 4) & 0xF               # extraire bits 7..4
x | (1 << 3), x & ~(1 << 3)  # mettre / effacer bit 3
f"{x:08b}", f"{x:04X}"       # binaire / hexa sur n
def signe(x, w):             # complément à 2
    return x - (1 << w) if x >> (w - 1) else x
x.bit_length(), bin(x).count("1")
x.to_bytes(4, "little"), int.from_bytes(b, "big")
import struct; struct.pack("<If", 7, 1.5)
```

>_ Environnement

```
python3 -m venv .venv        # env. isolé
source .venv/bin/activate    # (Windows : Scripts)
pip install numpy matplotlib
pip freeze > requirements.txt
python3 -i script.py         # console après
breakpoint()                 # débogueur pdb
```

Pièges

Liste de listes	`[[0]*3]*3` répète la même ligne : passer par une compréhension.
Flottants	`0.1 + 0.2 != 0.3` : comparer avec `math.isclose`.
is ou ==	`is` teste l'identité : seulement pour `None`.
Itérer et modifier	retirer d'une liste pendant un `for` saute des éléments : itérer sur une copie.

Défaut mutable : bloc Fonctions · alias : bloc Listes (recto).